

App Dev Project Report

1. Student Details

Name: Parvez Khan

Roll Number: 25f1000244

Email: 25f1000244@ds.study.iitm.ac.in

About Me: I am a student at the IIT Madras BS Degree program with a strong passion for full-stack development, software architecture, and building production-ready applications. I enjoy creating platforms that solve real-world problems using clean code and industry best practices.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To design and develop a comprehensive campus placement management system that connects students, hiring companies, and administrators. The platform aims to streamline job postings, student applications, profile management, and administrative approvals into one unified interface.

Approach:

The application was built using a strongly typed, decoupled architecture. The frontend is a Single Page Application (SPA) built with Vue.js 3 to provide a seamless, reactive user experience. The backend is a modular REST API built with Flask, utilizing SQLAlchemy for database operations. For heavy operations, Celery and Valkey (Redis) were integrated to handle background jobs like CSV exports and automated email notifications, while server-side API caching was implemented to optimize database load.

3. AI/LLM Declaration

I used AI tools strictly as a mentor and guide. I used AI to help me understand complex architectural concepts (like layered MVC architecture, configuring Celery workers, and JWT flows) and to seek advice on industry standards and best practices.

The extent of AI/LLM usage is around 10–15%, limited purely to conceptual guidance, code structure suggestions, and debugging hints. All core implementation logic, code writing, UI development, and system integration were built entirely manually by me.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Vue.js 3	Core frontend framework (Composition API)
Vuex & Vue Router	State management and client-side routing
Flask	Core backend REST API framework
SQLAlchemy	Object Relational Mapper for database interactions
SQLite	Lightweight local database for storing user data
Celery & Valkey (Redis)	Message broker and background task workers
Flask-JWT-Extended	Secure user authentication and session handling
Flask-Caching	Server-side caching for high-traffic endpoints

5. Database Schema / ER Diagram

Core Tables:

- **User** — stores authentication credentials and roles (id, email, password, role, status)
- **Student** — stores student academic profiles (id, user_id, name, branch, cgpa, expected_graduation_year, resume_file)
- **Company** — stores company profiles (id, user_id, name, industry, location, hr_contact)
- **PlacementDrive** — stores job postings (id, company_id, job_title, job_description, salary, min_cgpa, status)
- **Application** — maps students to drives (id, student_id, drive_id, status)

Relationships:

- **One-to-One** → User to Student / Company
- **One-to-Many** → Company to PlacementDrive
- **One-to-Many** → Student to Application
- **One-to-Many** → PlacementDrive to Application

(Insert ER diagram image here – refer to docs/ppa-schema-and-flow.png)

6. API Resource Endpoints (Sample)

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user and generate JWT token
/api/student/dashboard	GET	Fetch dashboard metrics and approved drives
/api/company/drive	POST	Post a new placement drive
/api/company/applications/<id>	GET	Fetch all student applications for a drive
/api/admin/approve_company/<id>	POST	Admin approval for newly registered companies

Note: Complete API definitions are available in the codebase.

7. Architecture and Features

Architecture Overview:

- **Frontend (/client)** – Component-based Vue.js architecture with centralized state management.
- **Backend (/server)** – Modular Flask architecture utilizing Controllers, Services, and Repositories.
- **Workers (/server/workers)** – Celery tasks for asynchronous execution.

Implemented Features:

- Multi-Role RBAC Authentication (Student, Company, Admin).
- Admin dashboard for moderating companies, drives, and tracking overall portal statistics.
- Student portal for resume uploading and one-click applications.
- Company portal for reviewing candidate profiles and updating application statuses.
- Secure password hashing via Werkzeug.
- Application-level caching for read-heavy routes.

Additional Advanced Features:

- **Asynchronous CSV Exports:** Non-blocking data exports for students and companies via Celery.
- **Automated Email Notifications:** Scheduled jobs (Celery Beat) sending HTML emails for interview reminders and monthly placement analytics.

8. Video Presentation

Drive Link:  ppa_demo.mp4